

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Hazardy
- 2 Třístavový výstup a sběrnice
- 3 ALU, registr, paměť

Téma

- 1 Hazardy
- 2 Třístavový výstup a sběrnice
- 3 ALU, registr, paměť

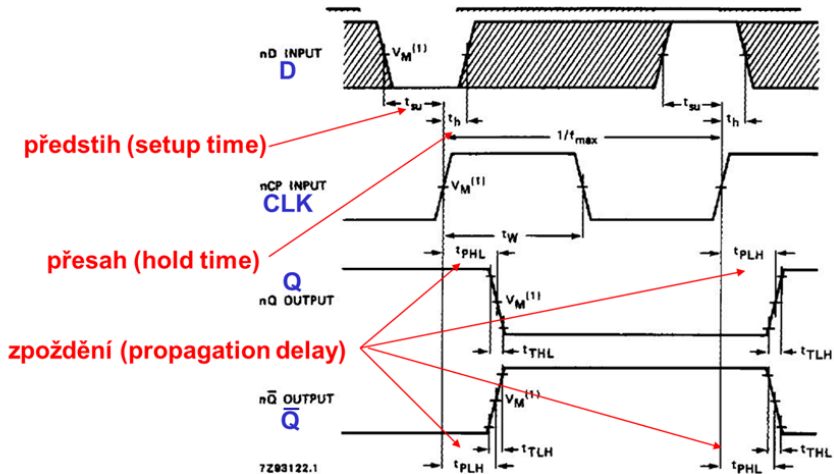
Reálné vlastnosti logických IO

AC CHARACTERISTICS FOR 74HCT

GND = 0 V; $t_r = t_f = 6$ ns; $C_L = 50$ pF

SYMBOL	PARAMETER	T _{amb} (°C)								UNIT	TEST CONDITIONS	
		74HCT									V _{CC} (V)	WAVEFORMS
		+25			-40 to +85		-40 to +125					
		min.	typ.	max.	min.	max.	min.	max.				
t _{PHL} / t _{PLH}	propagation delay nCP to nQ, nQ		18	35		44		53	ns	4.5	Fig.6	
t _{PHL} / t _{PLH}	propagation delay nSD to nQ, nQ		23	40		50		60	ns	4.5	Fig.7	
t _{PHL} / t _{PLH}	propagation delay nRD to nQ, nQ		24	40		50		60	ns	4.5	Fig.7	
t _{THL} / t _{TLH}	output transition time		7	15		19		22	ns	4.5	Fig.6	
t _W	clock pulse width HIGH or LOW	18	9		23		27		ns	4.5	Fig.6	
t _W	set or reset pulse width LOW	16	9		20		24		ns	4.5	Fig.7	
t _{rem}	removal time set or reset	6	1		8		9		ns	4.5	Fig.7	
t _{su}	set-up time nD to nCP	12	5		15		18		ns	4.5	Fig.6	
t _h	hold time nCP to nD	3	-3		3		3		ns	4.5	Fig.6	
f _{max}	maximum clock pulse frequency	27	54		22		18		MHz	4.5	Fig.6	

Reálné vlastnosti logických IO - D KO



Hazard - definice

Jde o nežádoucí přechodný děj, kdy výstup dočasně neodpovídá nastavené kombinaci vstupů.

Statický Přechod mezi shodnými stavy (oba v log. 1 nebo 0) logické funkce způsobí krátkodobou změnu výstupu na opačný stav.

Dynamický Přechod mezi stavy logické funkce způsobí několikanásobnou změnu výstupu.

Hazardy obvykle dělíme na dva druhy podle toho, zda:

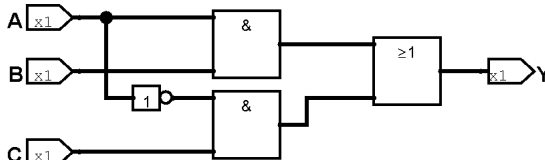
- 1 vzniká při změně jedné proměnné (sousední pozice v Karnaughově mapě), nebo
- 2 při současné změně více proměnných.

Statický hazard 1. druhu

	<i>CB</i>			
	00	01	11	10
0			1	1
1		1	1	

$$Y = \bar{A}C + AB$$

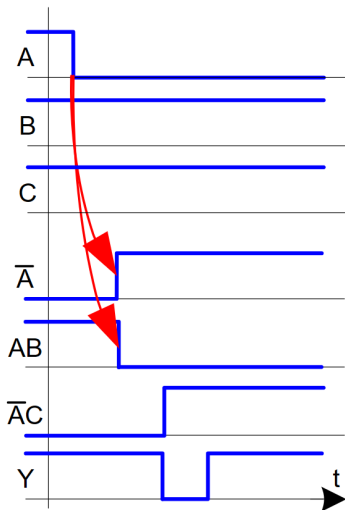
Invertor vytváří zpoždění



Problematický přechod je změna A při současném (B, C) = 1

Statický hazard 1. druhu - časový diagram

Signály:



Předpokládáme stejná zpoždění t_d na všech hradlech:

- ❶ Signál $\bar{A}$ je zpožděn za signálem A o t_d ,
- ❷ stejné zpoždění má i signál AB a ten tlačí stav výstupu Y do logické 0,
- ❸ logická úroveň $\bar{A}C = 1$ je vyhodnocena se zpožděním $2 \times t_d$,
- ❹ výstup Y se vrátí do logické 1 až po příchodu hodnoty $\bar{A}C = 1$, tj. na výstupu je dočasně generována nízká úroveň po dobu zpoždění t_d .

Statický hazard 1. druhu - redundance

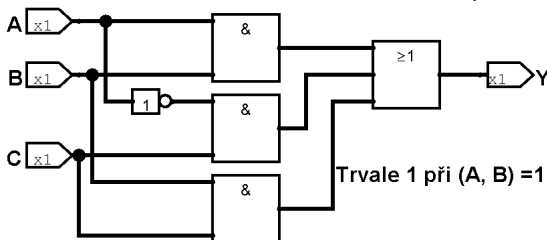
Truth table for function Y with inputs A and B (columns) and output Y (rows). The columns are labeled CB with values 00, 01, 11, 10. The rows are labeled A with values 0 and 1.

	00	01	11	10
0			1	1
1		1	1	

Groupings (Karnaugh map):

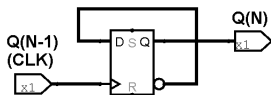
- Group 1 (Orange): AB (cells 11, 10)
- Group 2 (Pink): BC (cells 11, 01)
- Group 3 (Green): AC (cells 01, 11)

$$Y = \bar{A}C + AB + BC$$



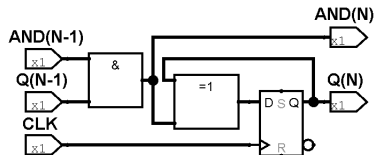
Rozdíly mezi asynchronním a synchronním čítačem

Asynchronní čítač



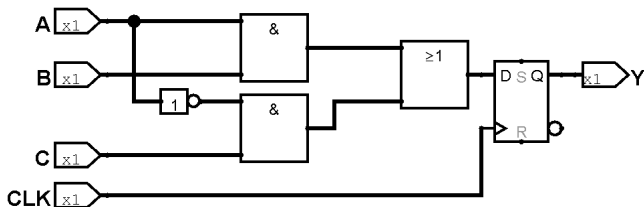
- + Obsahuje jen jeden typ hradla (KO),
- + velmi jednoduchá realizace,
- výstupy mají vzájemné zpoždění, navazující obvody mohou vykazovat hazardy.

Synchronní čítač



- Obsahuje nejméně dva typy hradel (JK KO + AND),
- složitější realizace,
- +++ všechny výstupy se změní v jeden okamžik hrany **CLK**.

Statický hazard 1. druhu - synchronizace



Předpoklad:

- Vstupy jsou synchronizovány na hranu **CLK** $\Rightarrow$ mění se všechny současně.

Výsledek:

- Výsledek je zapsán s hranou **CLK** až po odeznění hazardu,
- zvyšujeme zpoždění získání příslušného výsledku o 1 periodu **CLK**.

Co z toho všeho plyne pro praxi

- Vždy realizujeme synchronní obvody - platí i (hlavně) v hradlových polích,
- výsledky kombinační logiky synchronizujeme na hranu hodin pomocí klopných obvodů (např. D KO),
- čtení vstupů také synchronizujeme (vzorkujeme) pomocí D KO,
- výsledek logiky má vždy definované zpoždění v počtu period hodinového cyklu.

Téma

- 1 Hazardy
- 2 Třístavový výstup a sběrnice**
- 3 ALU, registr, paměť

Sběrnice

Soustava vodičů propojující mezi sebou dva nebo více elektronických systémů:

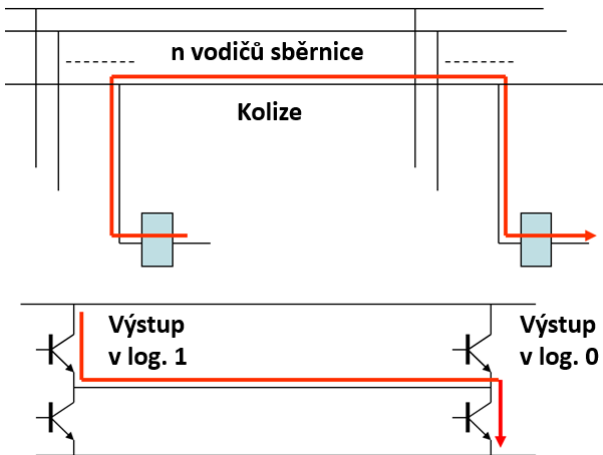
- Napájecí sběrnice,
- adresová sběrnice,
- datová atd.

Rozlišujeme dva druhy zařízení na sběrnici:

- vysílač** - nastavuje a udržuje signál na sběrnici,
- přijímač** - čte signál na sběrnici a reaguje na něj.

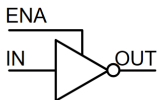
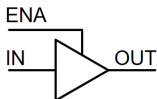
Sběrnice může mít několik vysílačů. Řešíme tak problém kolize výstupů, protože nelze připojit dva a více výstupů dohromady na jeden vodič.

Sběrnice - kolize výstupů



Třístavový buffer

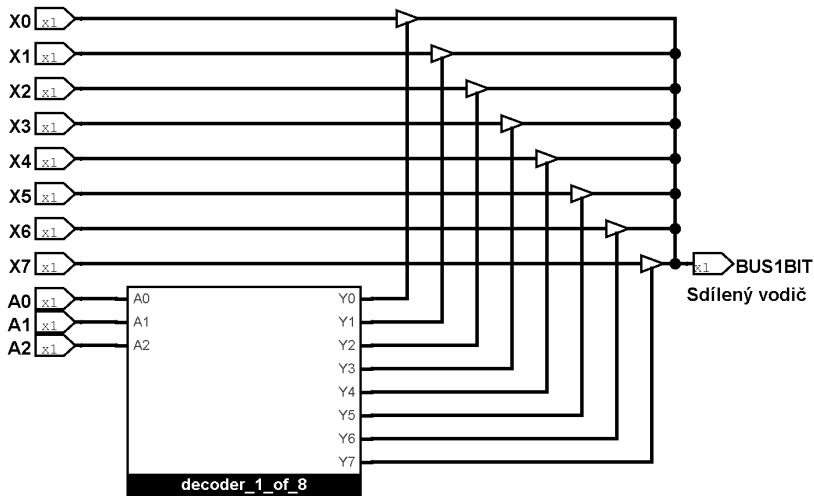
Symbol:



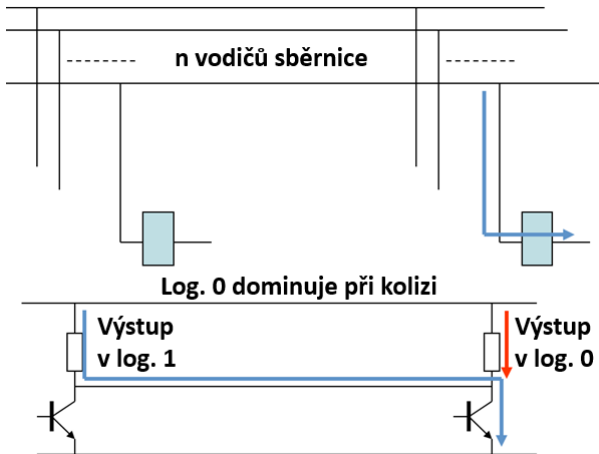
- Kromě dvou výstupních stavů logické 0 a logické 1 nabízí stav vysoké impedance,
- stavy se obvykle značí 0, 1, Z nebo L, H, Z,
- lze se setkat i s verzí s invertorem,
- na místě třístavového výstupu lze někdy použít hradla s otevřeným kolektorem.

<i>ENA</i>	<i>IN</i>	<i>OUT</i>
0	0	<i>Z</i>
0	1	<i>Z</i>
1	0	0
1	1	1

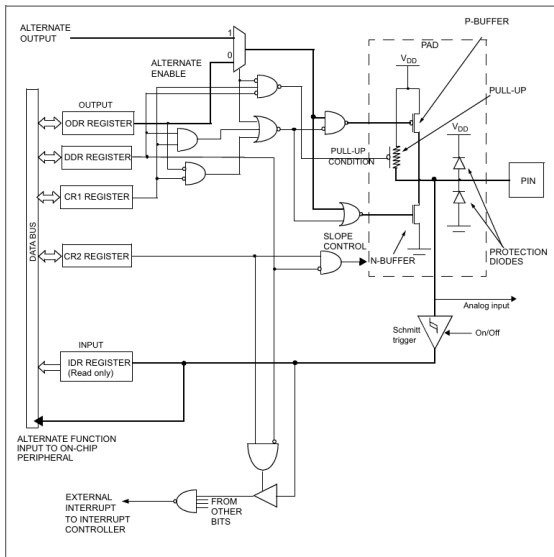
Multiplexer pro sběrnici



Sběrnice - otevřený kolektor



Příklad výstupu mikroprocesoru STM8S208



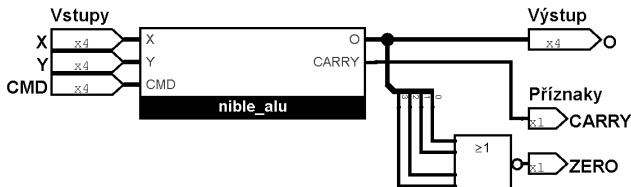
Téma

- 1 Hazardy
- 2 Třístavový výstup a sběrnice
- 3 ALU, registr, paměť**

Aritmeticko-logická jednotka (ALU)

- Je základní součástí procesoru:
 - provádí matematické operace (sčítání, odčítání, násobení, ...),
 - logické operace (AND, OR, NOT, ...),
 - bitové posuny a rotace,
 - pracuje s daty v **registrech** (STM8 - **akumulátor**).
- Konkrétní typ operace je určen **řadičem instrukcí**.
- Operandů jsou do registrů načítány po sběrnici z paměti a výsledek operací je ukládán do určeného registru (STM8 - **akumulátor**).
- Stav výsledku je signalizován příznakovými bity:
 - **C** ... přenos,
 - **Z** ... nulový výsledek,
 - **N** ... záporný výsledek atd.

ALU - příklad



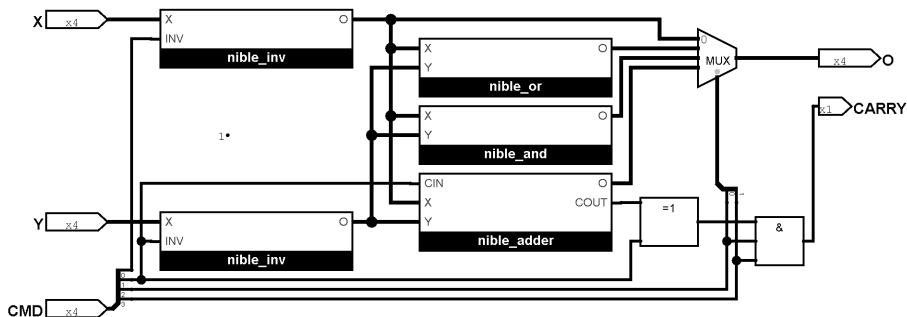
Proměnné:

- **X, Y** ... operandy,
- **CMD** ... volba operace,
- **O** ... výstup,
- **C, Z** ... příznakové bity.

Logické operace doplňuje možnost negace operandů pomocí bitů 0 a 1 v **CMD**.

CMD	Výstup
0000	X
0001	$\bar{X}$
0100	X or Y
1000	X and Y
1100	$X + Y$
1110	$X - Y$

ALU - příklad, vnitřní uskupení



- Čistě kombinační logika,
- **AND** a **OR** provádějí operaci mezi příslušnými bity, tj. **X0** s **Y0**, **X1** s **Y1**, **X2** s **Y2** atd.

ALU - Počátek vzhledu instrukce

Návrh Instrukce ve strojovém kódu = binární zápis

CMD	1. OPERAND	2. OPERAND
------------	-------------------	-------------------

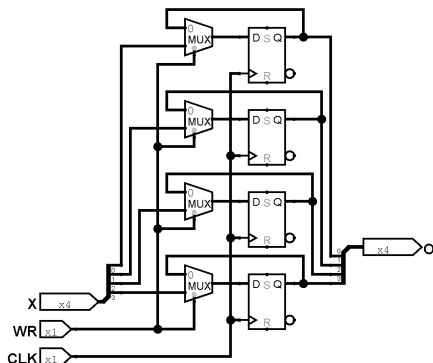
- Zápis obou operandů v instrukci prodlužuje její délku,
- výhodnější je zapsat nejdříve první operand do paměti ALU a pak následně provést požadovanou operaci s přenesením druhého operandu.

Registr:

- Velmi rychlá malá paměť (8, 16, 32 bitů), která je přímo součástí procesoru,
- není trvalou paměť, po ztrátě napájení dojde k jejímu vymazání,
- často plní specializovaný účel u konkrétní periferie, jako například vstup i výstup aritmetické jednotky.

Registr - příklad

Ukázka:

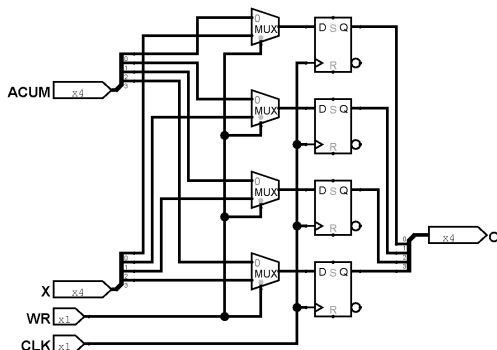


- 4 bit registr,
- synchronní zápis na hranu hodin,
- zápis se povoluje $WR = 1$,
- toto je logická reprezentace, nejedná se o skutečné provedení uvnitř CPU.

CLK	WR	OUT
↑	0	OUT
↑	1	X

Registr - akumulátor příklad

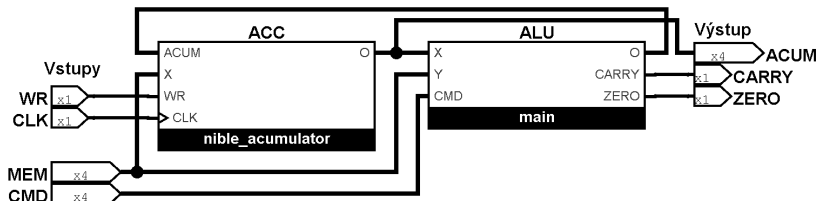
Ukázka:



- Zpětná vazba pro uchování hodnoty je jako vstup,
- jinak je zapojení stejné jako předchozí.

Vstup **ACUM** použijeme pro zápis výsledku z **ALU**.

ALU s akumulátorem



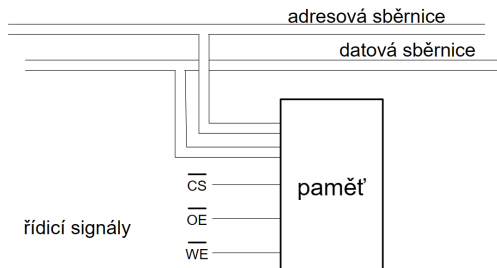
- Registr **ACC** tvoří jak jeden z operandů, tak cílovou paměť pro uložení výsledku.
- Druhý operand se načítá ze sběrnice (**MEM**), kde je buď obsah paměti nebo jiného registru.
- Každá **ALU** má registr s podobnou funkcí. Nicméně, název registru akumulátor (nebo střadač) je specifický pro procesory STM8.

Ukázka aritmetických a logických instrukcí STM8

základní instrukce																															
katedra elektrotechnologie ČVUT FEL 09.03.2011 -kk-																						priznaky					vysvětlení				
		A	XL	XH	YL	YH	CC	#d	a	aa	(X)	(a,X)	(aa,X)	(Y)	(a,Y)	(aa,Y)	(a,SP)	[a,w]	[aa,w]	[(a,w),X]	[(aa,w),X]	[(a,w),Y]	I1	I0	V	H		N	Z	C	
aritmetika	CLR	☐	-	1/1	-	-	-	-	-	1/2	1/4	1/1	1/2	1/4	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	0	1	-	☐ ← 0	
	NEG	☐	-	1/1	-	-	-	-	-	1/2	1/4	1/1	1/2	1/4	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	-	N	Z	C	☐ ← (not□)*1
	INC	☐	-	1/1	-	-	-	-	-	1/2	1/4	1/1	1/2	1/4	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	-	N	Z	-	☐ ← ☐ + 1
	DEC	☐	-	1/1	-	-	-	-	-	1/2	1/4	1/1	1/2	1/4	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	-	N	Z	-	☐ ← ☐ - 1
	ADD	A, ☐	-	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	H	N	Z	C	A ← A + ☐
	ADD	SP, ☐	-	-	-	-	-	-	2/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	SP ← SP + ☐
	ADC	A, ☐	-	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	H	N	Z	C	A ← A + ☐ + C
	SUB	A, ☐	-	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	-	N	Z	C	A ← A - ☐
	SUB	SP, ☐	-	-	-	-	-	-	2/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	SP ← SP - ☐
	SBC	A, ☐	-	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	-	N	Z	C	A ← A - ☐ - C
logika	MUL	X,A Y,A	-	-	4/1	4/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-	0	X ← XL * A Y ← YL * A		
	DIV	X,A Y,A	-	-	2-17/1	2-17/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	Z	X ← X/A Y ← Y/A ; A ← zbytek	
	CP	A, ☐	-	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	V	-	N	Z	C	A - ☐
	TNZ	☐	-	1/1	-	-	-	-	-	1/2	1/4	1/1	1/2	1/4	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	-	N	Z	-	test ☐
	CPL	☐	-	1/1	-	-	-	-	-	1/2	1/4	1/1	1/2	1/4	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	-	N	Z	1	☐ ← not. ☐
	AND	A, ☐	-	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	-	N	Z	-	A ← A. and. ☐
logika	OR	A, ☐	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	-	N	Z	-	A ← A. or. ☐	
	XOR	A, ☐	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	-	N	Z	-	A ← A. xor. ☐	
	BCP	A, ☐	-	-	-	-	-	1/2	1/2	1/3	1/1	1/2	1/3	1/2	1/3	1/4	1/2	4/3	4/4	4/3	4/4	4/3	-	-	-	-	N	Z	-	A. and. ☐	

- Akumulátor je součástí všech aritmetických i logických instrukcí assembleru STM8.

Paměť



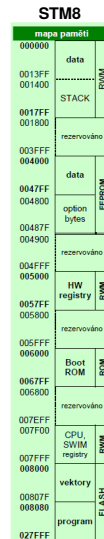
- Registry jsou nevhodné pro záznam většího množství dat, kvůli relativní složitosti paměťové jednotky (velikost, integrace),
- paměťová buňka je výrazně jednodušší,
- pro její obsluhu využíváme:
 - adresovou sběrnici pro nastavení buňky s daty,
 - datovou sběrnici pro zápis a čtení obsahu buňky a
 - řídicí signály pro kontrolu zápisu, přístupu na sběrnici apod.

Paměť - STM8

- Paměť dělíme na:
 - trvalou (nevolatilní), která uchovává data i po ztrátě napájení a
 - dočasnou (volatilní), která data ztratí se ztrátou napájení.

V případě procesorů řady STM8 je k dispozici paměť:

- **ROM, FLASH** - trvalá paměť určená pro zápis programu, přepisuje se po blocích.
- **EEPROM** - trvalá paměť určená pro zálohování dat programu, přepisuje se po jednotkách.
- **RWM = RAM** - volatilní paměť určená pro práci s daty, velmi rychlá.



Následující přednáška

- Architektura CPU,
- Mikroprocesor STM8S208RB,
- Instrukce a strojový kód,
- CISC a RISC

Děkuji za pozornost